



UNIVERSITÀ DEGLI STUDI DI VERONA
DIPARTIMENTO DI INFORMATICA

Corso di Laurea **triennale** in **Informatica**

**Studio e Utilizzo di framework per la progettazione e lo
sviluppo di una applicazione Web per la gestione di un
archivio di libri.**

Relatore

PROF. ELISA QUINTARELLI

Candidato

MATTEO ADAMO

Matricola: VR471358

Sessione di Laurea: DICEMBRE 2025

Anno Accademico 2024/2025

Indice

Abstract	iii
1 Requisiti dati	1
1.1 Obiettivi funzionali	1
1.2 Requisiti non funzionali	1
1.3 Scelta del framework	2
2 Motivazioni	3
2.1 Panoramica	3
3 Progettazione	9
3.1 Architettura generale	9
3.2 Schema ER e modello concettuale	10
3.2.1 Modello Entità-Relazione	10
3.2.2 Script SQL di creazione del Database con PostgreSQL	11
4 Codici realizzati	13
4.1 Lato backend : app.js	13
4.1.1 Importazione librerie:	13
4.1.2 Creazione dell'app Express e configurazione della porta:	14
4.1.3 Configurazione connessione PostgreSQL:	14
4.1.4 Configurazione Multer per salvataggio file PDF:	15
4.1.5 Middleware Express:	15
4.1.6 Endpoint per la pagina principale:	16
4.1.7 Configurazione Multer per immagini (OCR):	16
4.1.8 Endpoint /upload-pdf/:isbn:	16
4.1.9 Endpoint /upload per OCR:	17
4.1.10 Rotte del codice:	20
4.2 Lato frontend : index.html	21
4.2.1 Struttura iniziale della pagina (DOCTYPE, HTML e HEAD)	21

4.2.2	Definizione degli stili CSS	22
4.2.3	Header con il titolo dell'applicazione	24
4.2.4	Contenitore principale e pulsanti di navigazione	24
4.2.5	Script JavaScript: logica e interattività della pagina	26
5	Modelli Deep per il riconoscimento di codici ISBN	28
5.1	Introduzione al Deep Learning	28
5.2	Riconoscimento Ottico dei Caratteri (OCR)	29
5.2.1	Tesseract OCR	30
5.3	Diagnosi delle Criticità	30
5.3.1	Approccio con Rete Neurale Personalizzata	30
5.3.2	Performance di Tesseract OCR	34
6	Discussione	36
6.1	Introduzione	36
6.2	Strategie di miglioramento proposte per OCR	36
6.3	Sicurezza e Autenticazione Avanzata	37
6.4	Algoritmi di Ricerca e Raccomandazione	37
6.5	User Experience e Interfaccia	38
6.6	Conclusioni e Sviluppi Futuri	39

Abstract

Questo lavoro di tesi descrive lo studio e l'implementazione di un sistema integrato per la gestione e la ricerca di bibliografia tecnica e manualistica attraverso interfacce web. Il progetto si articola in una fase di analisi degli obiettivi fondamentali, seguita da una disamina dettagliata delle scelte tecnologiche effettuate durante lo sviluppo.

Vengono approfondite le motivazioni alla base della selezione di framework, librerie, strategie di sicurezza per la gestione degli account e sistemi di database, confrontando le soluzioni adottate con alternative potenzialmente percorribili. L'analisi include l'esame di porzioni significative di codice, sia lato frontend che backend, con commenti puntuali che illustrano il funzionamento e le logiche implementative.

Il lavoro prosegue con un'introduzione ai concetti fondamentali del Deep Learning e dei sistemi di Riconoscimento Ottico dei Caratteri (OCR), applicati al contesto specifico del progetto. Particolare attenzione viene dedicata all'analisi critica degli errori riscontrati e delle problematiche emerse durante le fasi di sviluppo e testing.

Infine, vengono discusse le strategie risolutive adottate e le prospettive di miglioramento futuro, delineando un percorso di evoluzione del sistema che possa beneficiare delle considerazioni emerse dall'esperienza pratica. Lo studio dimostra come l'approccio metodologico basato su analisi comparative e validazione empirica abbia permesso di raggiungere gli obiettivi prefissati, pur aprendo a nuovi interessanti sviluppi nel campo dell'elaborazione intelligente di documenti.

Capitolo 1

Requisiti dati

1.1 Obiettivi funzionali

L'applicazione ha come obiettivo principale la raccolta, l'organizzazione e la ricerca di manuali e bibliografia tecnica. Le funzionalità principali previste includono:

- gestione account utente (registrazione, login, ruoli amministratore/utente);
- caricamento e gestione di risorse (libri, manuali, copertine, PDF);
- estrazione automatica di metadati tramite OCR dalle immagini caricate;
- ricerca full-text e per filtri (autore, tag, categoria, anno);
- librerie personali e libreria globale amministrata centralmente;
- condivisione e riutilizzo di entry tra librerie diverse.

1.2 Requisiti non funzionali

- sicurezza: password cifrate con algoritmo robusto (es. bcrypt);
- scalabilità: scelta di PostgreSQL per supportare grandi volumi di dati e query complesse;
- portabilità: architettura separata frontend/backend che consenta evoluzioni (es. app mobile);
- affidabilità OCR: il sistema deve registrare la confidenza dell'OCR e prevedere intervento umano per correzioni.

1.3 Scelta del framework

Per lo sviluppo dell'applicazione web/mobile oggetto del tirocinio, è stata adottata la scelta di utilizzare **Express.js**, un framework leggero e flessibile basato su Node.js. Questa decisione è stata motivata da diversi fattori:

- **Compatibilità con applicazioni web e mobile:** Express.js consente di creare API efficienti e facilmente integrabili con frontend web e mobile, caratteristiche fondamentali per un'applicazione pensata per la gestione e ricerca di manuali accessibile da diversi dispositivi.
- **Rapidità di sviluppo:** grazie alla sua leggerezza e all'ampio ecosistema di pacchetti Node.js, Express permette di sviluppare velocemente funzionalità complesse, riducendo i tempi di prototipazione.
- **Scelta rispetto a Flask:** Seppur Flask, in quanto micro-framework Python, costituisca una valida alternativa, specialmente in scenari che richiedono un'alta intensità di calcolo per l'integrazione di librerie scientifiche native Python, la scelta per l'architettura di questo progetto è ricaduta su Node.js con Express.js per ragioni architetturali e di coerenza dello stack tecnologico.

Il sistema, basato sulla ricezione di file immagine e sulla loro successiva elaborazione OCR, è intrinsecamente I/O-intensive. Il modello event-driven e non bloccante di Node.js si è rivelato superiore per la gestione efficiente e concorrente di queste operazioni asincrone, garantendo una maggiore reattività del server rispetto all'approccio bloccante di molti framework basati su Python (a meno di ricorrere a configurazioni specifiche come `gevent` o `asyncio`).

Inoltre, la scelta di un ecosistema Full-Stack JavaScript ha permesso di ottimizzare la curva di apprendimento e la gestione delle dipendenze, mantenendo l'intero sviluppo su un unico linguaggio (JavaScript/TypeScript) e facilitando la manutenibilità e la coerenza tra frontend e backend.

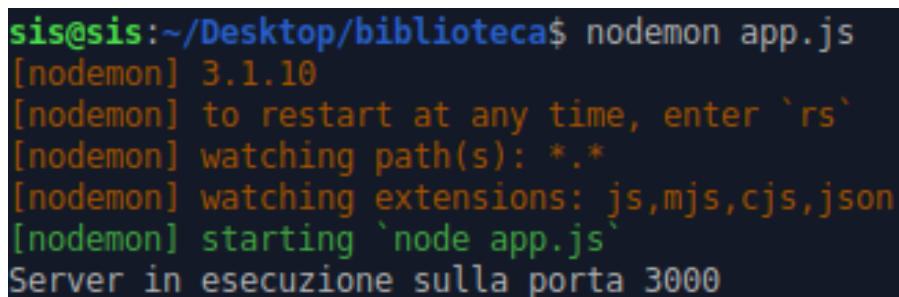
Capitolo 2

Motivazioni

2.1 Panoramica

Per soddisfare i requisiti sono state selezionate le seguenti tecnologie principali, motivate di seguito:

- **Node.js + Express.js:** adottati per realizzare alcune componenti del backend e fornire API REST necessarie alla comunicazione tra i diversi moduli dell'applicazione. La scelta è stata dettata dalla mia familiarità con l'ecosistema JavaScript e dalla possibilità di integrare facilmente librerie lato server, come Tesseract.js, per la gestione di operazioni specifiche. Express.js, in particolare, ha garantito una struttura chiara delle rotte e una gestione semplificata delle richieste HTTP.



```
sis@sis:~/Desktop/biblioteca$ nodemon app.js
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Server in esecuzione sulla porta 3000
```

Figura 2.1: Eseguire il server con il comando *nodemon* sulla porta 3000.

- **PostgreSQL:** Durante il corso di *Basi di Dati – Laboratorio* ho avuto l'opportunità di utilizzare il database relazionale PostgreSQL, approfondendone le funzionalità principali e acquisendo dimestichezza con la gestione dei dati attraverso il linguaggio SQL. Questa esperienza formativa si è rivelata particolarmente utile durante il mio tirocinio,

in cui ho scelto di adottare PostgreSQL come sistema di gestione del database per l'applicazione su cui stavo lavorando. La scelta è stata motivata dalla solidità e affidabilità di PostgreSQL, dalla sua conformità agli standard SQL e dalla disponibilità di estensioni e funzionalità avanzate che ne ampliano le possibilità di utilizzo. La struttura robusta del motore di database, unita alle capacità di gestione sicura delle transazioni e all'efficienza nelle interrogazioni complesse, ha garantito un'archiviazione e un recupero dei dati affidabile e performante. Inoltre, la familiarità acquisita durante il corso mi ha permesso di integrare rapidamente il database con il resto dell'applicazione, riducendo i tempi di sviluppo e aumentando la qualità complessiva del progetto.

```
sis@sis:~/Desktop/biblioteca$ psql -U postgres -d biblioteca
Password for user postgres:
psql (14.18 (Ubuntu 14.18-0ubuntu0.22.04.1))
Type "help" for help.

biblioteca=#
```

Figura 2.2: Accedere al mio database "biblioteca" tramite utente postgres.

– Analisi del mercato database e giustificazione della scelta

La selezione di PostgreSQL come sistema di database è supportata dalla sua posizione di mercato e affidabilità, come dimostrato dal DB-Engines Ranking ufficiale.

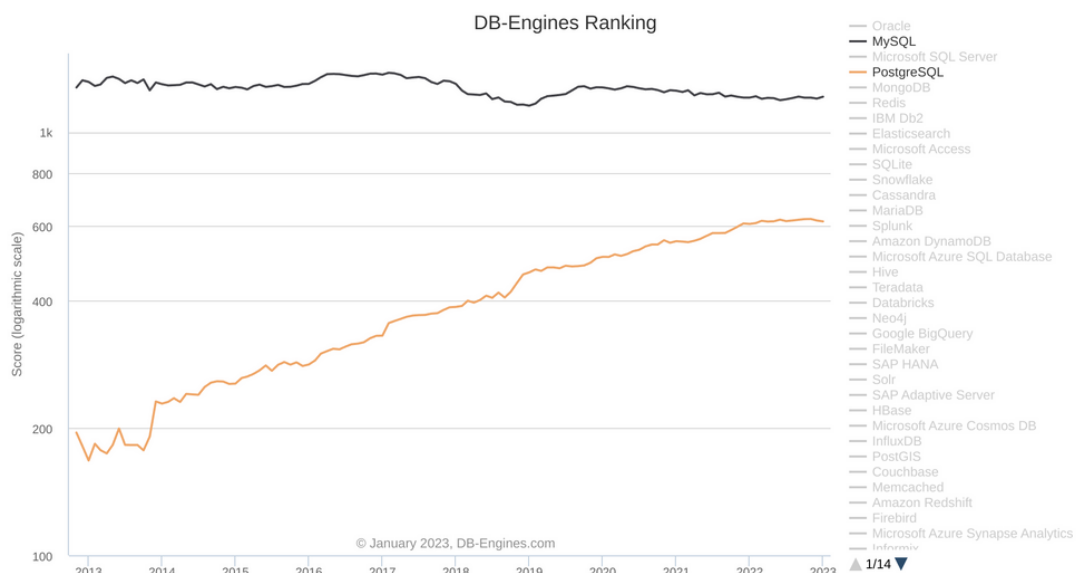


Figura 2.3: DB-Engines Ranking - Classifica mondiale dei sistemi di database (Gennaio 2023). PostgreSQL si posiziona al 4° posto assoluto, dimostrando la sua maturità e adozione su scala enterprise [19]

– Significato del DB-Engines Ranking

Il DB-Engines Ranking è considerato lo standard di riferimento per valutare la popolarità dei sistemi di database a livello globale. Il punteggio viene calcolato considerando:

- * Frequenza di menzioni sui siti web
- * Interesse generale nelle ricerche online
- * Discussioni sui social network e community tecniche
- * Offerte di lavoro che richiedono specifiche competenze
- * Profili professionali che menzionano la tecnologia

– Analisi della posizione di PostgreSQL

Come evidenziato nella Figura 2.3, PostgreSQL occupa la:

- * **4^a posizione assoluta** nella classifica generale
- * **2^a posizione** tra i database relazionali open-source
- * Posizione superiore a tecnologie enterprise come IBM Db2, Elasticsearch e Microsoft Access

Questa posizione di mercato garantisce:

- * **Supporto a lungo termine:** Community attiva e sviluppo continuo
- * **Disponibilità di risorse:** Documentazione completa e expertise diffusa
- * **Affidabilità enterprise:** Utilizzo in ambienti mission-critical
- * **Compatibilità ecosystem:** Integrazione con strumenti moderni

– Confronto con alternative considerate

Database	Ranking	Considerazioni
PostgreSQL	4°	Scelta finale - Maturo e feature-rich
MySQL	2°	Alternativa considerata, meno avanzato
MongoDB	12°	Scartato - Non ottimale per dati strutturati
SQLite	10°	Scartato - Non adatto per applicazioni multi-utente

Tabella 2.1: Confronto delle alternative di database basato sul DB-Engines Ranking

- **Tesseract.js (OCR):** la libreria è stata utilizzata per l'estrazione automatica di testo a partire da immagini, con l'obiettivo di digitalizzare contenuti senza dover inserire manualmente i dati. Nonostante la sua efficacia, durante lo sviluppo sono emersi alcuni limiti: immagini sfocate, caratteri molto piccoli o scarso contrasto tra testo e sfondo hanno reso complesso il riconoscimento, producendo risultati incompleti o imprecisi. Nel codice sviluppato non è stato implementato un vero pre-processing delle immagini, ma solo una fase di pulizia del testo riconosciuto. Questo ha funzionato in molti

casi, ma ha mostrato i suoi limiti con immagini di scarsa qualità. Un miglioramento futuro potrebbe consistere nell'aggiungere un pre-processing dell'immagine (ad esempio aumentando il contrasto o riducendo il rumore) per migliorare l'affidabilità del riconoscimento OCR.

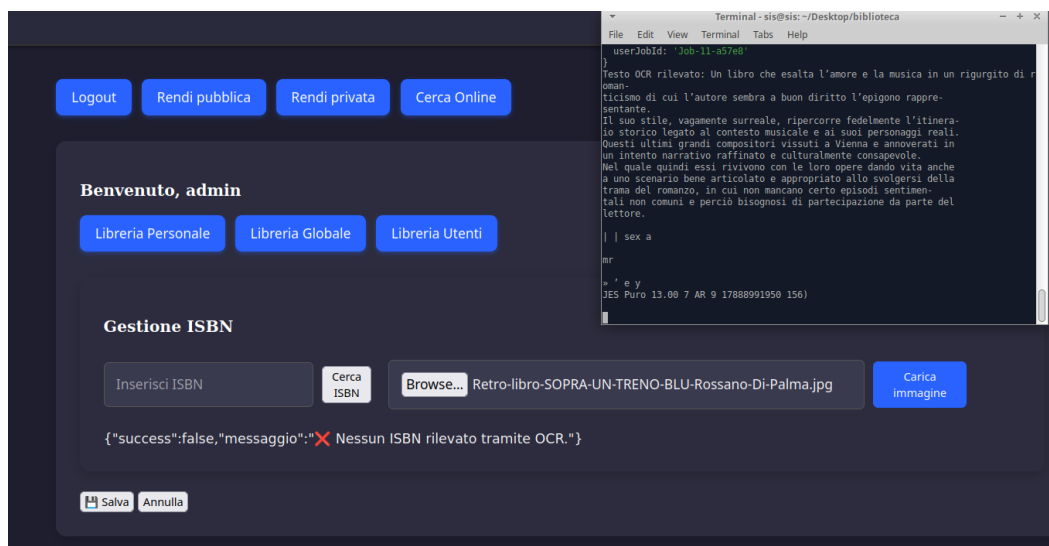


Figura 2.4: Errore in fase di lettura di OCR.

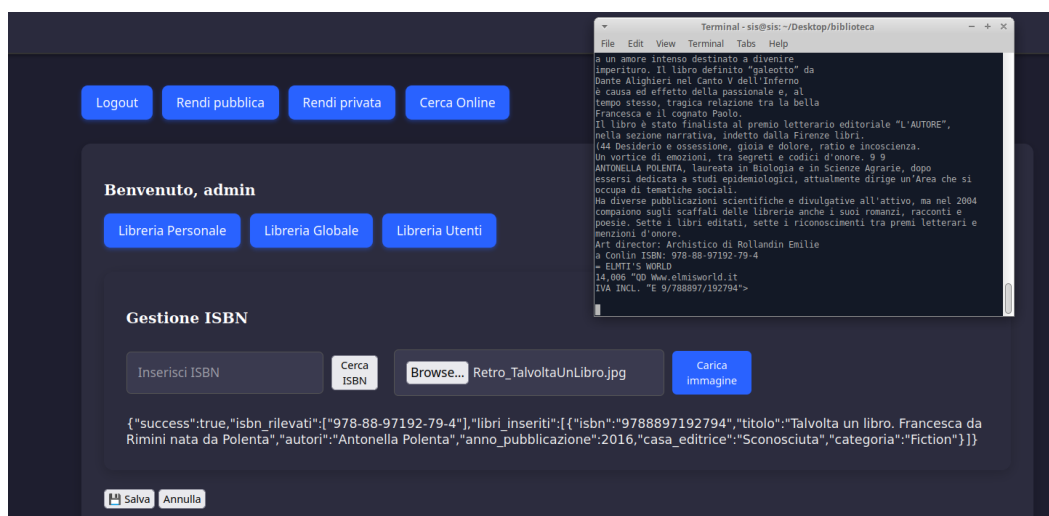


Figura 2.5: Successo di OCR.

- **Multer, dotenv, cors:** librerie di supporto utilizzate rispettivamente per la gestione sicura e strutturata dell'upload di file, per la configurazione centralizzata delle variabili d'ambiente e per l'abilitazione del *Cross-Origin Resource Sharing*. Multer è risultato essenziale per la gestione di file provenienti dagli utenti, permettendo di definire regole di salvataggio e filtri sui formati accettati. Dotenv ha facilitato la separazione tra codice e configurazioni sensibili, migliorando la sicurezza e la portabilità del progetto.

Infine, cors ha permesso di rendere accessibili le API in contesti distribuiti, garantendo compatibilità con applicazioni client ospitate su domini differenti.

- **bcrypt.js:** Nel corso *Sicurezza e Programmazione delle Reti* ho approfondito i principi e il funzionamento delle funzioni di hashing, comprendendone il ruolo nella protezione delle credenziali. In particolare, ho studiato le loro caratteristiche di non invertibilità e l'utilizzo del salt per garantire l'unicità degli hash e prevenire attacchi tramite tabelle pre-calcolate.

Durante il tirocinio, ho applicato queste conoscenze nell'utilizzo della libreria `bcryptjs` per la gestione sicura delle password. La libreria implementa internamente funzioni hash sicure con salt automatico e consente la verifica delle credenziali confrontando l'hash generato in fase di autenticazione con quello memorizzato. Ho inoltre adottato un'ulteriore misura di sicurezza, consistente nella concatenazione alla password di informazioni univoche, come la data e l'ora di creazione dell'account, per ottenere hash distinti anche in caso di password identiche.

Questa esperienza ha consolidato le competenze teoriche acquisite, permettendomi di comprenderne l'applicazione pratica in un contesto reale di sviluppo e sicurezza informatica.

Campo password con le password di ogni utente criptate con bcrypt

Colonna che tiene conto di quali utenti hanno l'account pubblico/privato

ID univoco per ogni utente

id	username	email	password	ispublic
10	davide98	davideadamo98@gmail.com	\$2b\$10\$Q2XYCJgHnV05WapyDX0pfe1PRUN4V2RUWXj0bYrDYXcWkQbFrZzum	f
12	Commercialisti	studiocommercialisti@gmail.com	\$2b\$10\$IGDibGCBs8M/k81c18BxFO5FCa1Lf55i9unRcte0vj2eeiQN8it2K	f
9	Mauro	mauroadamo61@gmail.com	\$2b\$10\$CIYNXD.MDT4t7niMSmMAu0TG0bUzus7md4nDPxe/XVI.yJVUYih7G	t
8	Matteo	matteoadamo@studenti.univr.it	\$2b\$10\$eCqw0oWofAp7cV0njV91.DDfNjrdIVedtIwDUmHRtvj4sTtwj99i	t
16	admin	admin	\$2b\$10\$znZh0.vWCikoyUR/.ITHLG.mycH0nKz8nBwLxo3AwHFNv0qGssWUe	t

Figura 2.6: Database Utente dove viene salvata la pw hashata

Quando si utilizza `bcrypt` (o `bcrypt.js` in JavaScript), la stringa salvata nel database ha un formato standard:

`$<versione>$<cost factor>$<salt+hash>`

- `$<versione>` : indica la versione dell'algoritmo `bcrypt` (es. 2a, 2b).

- **\$<cost factor>** : numero di cicli di elaborazione (potenza di due) utilizzati per generare l'hash; più alto è il valore, maggiore è la sicurezza ma anche il tempo di calcolo.
- **\$<salt+hash>** : blocco codificato in Base64 che contiene sia il *salt* (16 byte) sia l'hash vero e proprio (24 byte).

Esempio pratico

Consideriamo la prima riga del database:

\$2b\$10\$Q2XYCjhnV05WapvDX0pfeLPURNA4v2RUKXjDbYrDYvCkWQbFrZzum

Questo valore può essere scomposto come segue:

- **Versione:** \$2b\$
- **Cost factor:** 10 (equivale a $2^{10} = 1024$ iterazioni interne)
- **Salt + hash:** Q2XYCjhnV05WapvDX0pfeLPURNA4v2RUKXjDbYrDYvCkWQbFrZzum
 - * I primi 22 caratteri (Q2XYCjhnV05WapvDX0pfeL) rappresentano il *salt* in Base64.
 - * I restanti caratteri rappresentano l'hash della password combinata con il salt.

Questo formato garantisce che il *salt* sia sempre memorizzato insieme all'hash, permettendo la verifica senza doverlo salvare separatamente.

Capitolo 3

Progettazione

3.1 Architettura generale

Il sistema è progettato seguendo un'architettura three-tier che separa chiaramente le responsabilità tra presentazione, logica di business e gestione dei dati. Questa scelta architetturale garantisce modularità, manutenibilità e scalabilità dell'applicazione.

- *Frontend*: Il livello di presentazione è implementato attraverso una singola pagina applicativa basata su un file `index.html` per la struttura semantica, CSS per lo styling responsivo e JavaScript per l'interattività client-side. Questo approccio garantisce una esperienza utente fluida senza ricaricamenti completi di pagina durante le operazioni di ricerca e gestione delle risorse.
- *Backend*: Il livello applicativo è sviluppato in Node.js con il framework Express.js, che fornisce un set robusto di funzionalità per applicazioni web e API RESTful. Il server gestisce l'autenticazione degli utenti, l'elaborazione dei file caricati, l'integrazione con il motore OCR Tesseract.js e l'esposizione di endpoint JSON.
- *Database*: Il livello di persistenza dei dati utilizza PostgreSQL, un database relazionale open-source di classe enterprise. La scelta è motivata dal suo supporto nativo per query complesse, tipi di dati avanzati (JSONB) e meccanismi di concorrenza robusti, essenziali per garantire l'integrità dei dati bibliografici.

3.2 Schema ER e modello concettuale

3.2.1 Modello Entità-Relazione

Il modello Entità-Relazione (ER) è uno strumento concettuale utilizzato per rappresentare in modo astratto la struttura dei dati di un sistema informativo. Attraverso tale modello è possibile descrivere le informazioni rilevanti del dominio applicativo, individuando le entità principali, i loro attributi e le relazioni che intercorrono tra di esse.

L'obiettivo del modello ER è fornire una visione logica e coerente dei dati, indipendente dall'implementazione fisica, facilitando così la comprensione della realtà che il sistema deve gestire.

Inoltre, esso costituisce la base per la successiva progettazione logica del database, permettendo di tradurre il modello concettuale in uno schema relazionale concreto.

Nel contesto del presente progetto, la costruzione del modello ER ha permesso di analizzare e strutturare le informazioni fondamentali del dominio applicativo, garantendo una rappresentazione chiara e non ambigua dei dati e delle loro interdipendenze.

Lo schema, rappresentato nella Figura 3.1, definisce la struttura logica dei dati indipendentemente dall'implementazione fisica.

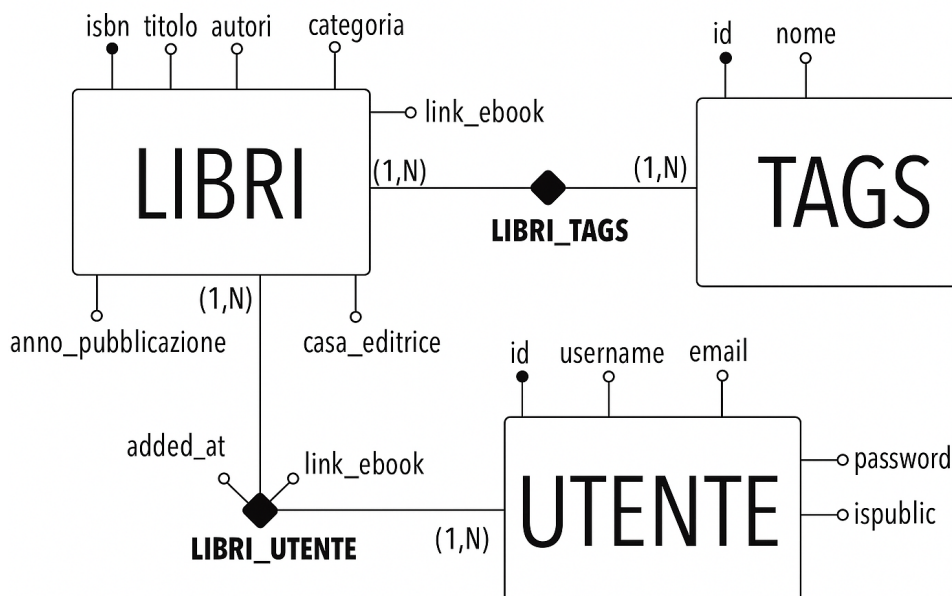


Figura 3.1

3.2.2 Script SQL di creazione del Database con PostgreSQL

```
-- Tabella: utente
CREATE TABLE utente (
    id SERIAL PRIMARY KEY,
    username varchar(50) NOT NULL UNIQUE,
    email varchar(100) NOT NULL UNIQUE,
    password text NOT NULL,
    ispublic boolean DEFAULT false
);

-- Tabella: libri
CREATE TABLE libri (
    isbn varchar(17) NOT NULL UNIQUE,
    titolo varchar(255) NOT NULL,
    autori varchar(255),
    anno_pubblicazione integer,
    casa_editrice varchar(255),
    categoria varchar(255),
    link_ebook text
);

-- Tabella: libri_utente
CREATE TABLE libri_utente (
    utente_id integer NOT NULL,
    isbn varchar(17) NOT NULL,
    added_at timestamp DEFAULT CURRENT_TIMESTAMP,
    link_ebook text,
    PRIMARY KEY (utente_id, isbn),
    FOREIGN KEY (utente_id) REFERENCES utente(id) ON DELETE CASCADE,
    FOREIGN KEY (isbn) REFERENCES libri(isbn)
);
```

```
-- Tabella: libri_tags
CREATE TABLE libri_tags (
    isbn text NOT NULL,
    tag_id integer NOT NULL,
    PRIMARY KEY (isbn, tag_id),
    FOREIGN KEY (isbn) REFERENCES libri(isbn) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

-- Tabella: tags
CREATE TABLE tags (
    id SERIAL PRIMARY KEY,
    nome text NOT NULL UNIQUE
);
```


Capitolo 4

Codici realizzati

4.1 Lato backend : app.js

4.1.1 Importazione librerie:

Importazione librerie

```
1 const express = require("express");
2 const { Pool } = require("pg");
3 const path = require("path");
4 const fetch = (...args) => import('node-fetch')
5   .then(({ default: fetch }) => fetch(...args));
6 const bcrypt = require("bcryptjs");
7 const cors = require("cors");
8 const bodyParser = require("body-parser");
9 const multer = require("multer");
10 const tesseract = require("tesseract.js");
11 const fs = require("fs");
12 require("dotenv").config({ path: "./.env" });
```

- **express:** Framework Node.js per gestire server, routing e middleware.
- **pg (Pool):** Client PostgreSQL per connettersi al database e gestire query in maniera efficiente tramite pool di connessioni.
- **path:** Gestione percorsi file cross-platform.
- **node-fetch:** Funzione fetch compatibile Node.js per chiamate HTTP/HTTPS asincrone.
- **bcryptjs:** Libreria per hashing password sicuro con salting.

- **cors:** Middleware per abilitare Cross-Origin Resource Sharing.
- **body-parser:** Middleware per parsing di JSON e form-urlencoded nelle richieste HTTP.
- **multer:** Middleware per gestione upload file multipart/form-data.
- **tesseract.js:** OCR basato su rete LSTM, permette riconoscimento testo da immagini lato server.
- **fs:** File system nativo per leggere, scrivere e cancellare file.
- **dotenv:** Carica variabili ambiente da file .env in process.env per sicurezza e configurazione.

4.1.2 Creazione dell'app Express e configurazione della porta:

App Express

```
1 const app = express();  
2 const port = process.env.PORT || 3000;
```

- **app = express():** Inizializza l'applicazione Express.
- **port = process.env.PORT || 3000:** Imposta porta di ascolto, prende valore da variabile ambiente o default 3000.

4.1.3 Configurazione connessione PostgreSQL:

Connessione PostgreSQL

```
1 const pool = new Pool({  
2   user: process.env.DB_USER,  
3   host: process.env.DB_HOST,  
4   database: process.env.DB_NAME,  
5   password: process.env.DB_PASS,  
6   port: process.env.DB_PORT,  
7 });
```

- **Pool:** Gestione efficiente delle connessioni al database con pooling per ridurre overhead.
- **Parametri:** Configurazione sicura tramite variabili ambiente per utente, host, database, password e porta.

- **Funzionamento:** Pool mantiene più connessioni aperte e le riutilizza per le query, migliorando performance in scenari con molte richieste.

4.1.4 Configurazione Multer per salvataggio file PDF:

Upload PDF

```
1 const pdfStorage = multer.diskStorage({
2   destination: (req, file, cb) => cb(null, 'uploads/'),
3   filename: (req, file, cb) => cb(null, Date.now() + path.extname(
4     file.originalname))
5 });
6 const uploadPDF = multer({ storage: pdfStorage });
```

- **diskStorage:** Specifica destinazione e nome file per upload sul server.
- **destination:** Cartella locale 'uploads/' dove salvare PDF.
- **filename:** Nome generato con timestamp + estensione originale per evitare collisioni.
- **uploadPDF:** Middleware Multer configurato per singolo file PDF.

4.1.5 Middleware Express:

Middleware Express

```
1 app.use(cors());
2 app.use(express.json());
3 app.use(express.static("public"));
4 app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
5 ;
```

- **cors():** Permette richieste cross-origin da frontend.
- **express.json():** Parsing automatico di body JSON in richieste HTTP.
- **express.static("public"):** Serve file statici come HTML, CSS, JS lato client.
- **express.static("/uploads"):** Espone cartella upload per accesso file caricati.

4.1.6 Endpoint per la pagina principale:

Pagina principale

```
1 app.get("/", (req, res) => {  
2   res.sendFile(path.join(__dirname, "views", "index.html"));  
3 });
```

- **app.get("/")**: Route HTTP GET per la homepage.
- **res.sendFile**: Invia file HTML statico come risposta.
- **path.join**: Garantisce compatibilità cross-platform nella costruzione del percorso file.

4.1.7 Configurazione Multer per immagini (OCR):

Upload Immagini OCR

```
1 const storage = multer.diskStorage({  
2   destination: (req, file, cb) => cb(null, "uploads/"),  
3   filename: (req, file, cb) => cb(null, Date.now() + path.extname(  
4     file.originalname)),  
5 });  
6 const upload = multer({ storage: storage });
```

- **diskStorage**: Configura cartella di salvataggio e naming per immagini.
- **upload**: Middleware Multer per upload singolo file immagine destinato all'OCR.

4.1.8 Endpoint /upload-pdf/:isbn:

Upload PDF per ISBN

```
1 app.post('/upload-pdf/:isbn', uploadPDF.single('pdf'), async (req,  
2   res) => {  
3   const { isbn } = req.params;  
4   const user_id = req.body.user_id;  
5  
6   if (!req.file) return res.status(400).json({ error: "Nessun file  
7     caricato." });  
8  
9   const link_ebook = `~/uploads/${req.file.filename}`;
```

```

9   try {
10     await pool.query(
11       `UPDATE libri_utente SET link_ebook = $1 WHERE isbn = $2 AND
        utente_id = $3`,
12       [link_ebook, isbn, user_id]
13     );
14     res.json({ success: true, link_ebook });
15   } catch (err) {
16     console.error("Errore salvando PDF:", err);
17     res.status(500).json({ error: "Errore durante l'upload." });
18   }
19 });

```

- **app.post('/upload-pdf/:isbn')**: Route POST per upload PDF associato a un ISBN specifico.
- **uploadPDF.single('pdf')**: Middleware Multer per singolo file PDF.
- **link_ebook**: Percorso file salvato per accesso successivo.
- **pool.query()**: Aggiorna record del libro utente nel database con link PDF.
- **Gestione errori**: Restituisce 400 se file mancante, 500 se errore database.

4.1.9 Endpoint /upload per OCR:

OCR Immagini

```

1 app.post("/upload", upload.single("image"), async (req, res) => {
2   if (!req.file) return res.status(400).send("Nessun file caricato.");
3
4   const imagePath = path.join(__dirname, "uploads", req.file.
        filename);
5
6   try {
7     const { data: { text } } = await tesseract.recognize(imagePath,
        "eng+ita", {
8       tesseract_char_whitelist: "0123456789Xx-",
9       logger: m => console.log(m),
10    });
11
12    const cleanedText = text

```

```

13     .replace-(/[=]/g, "-")
14     .replace(/[^\x20-\x7E]/g, '')
15     .replace(/\s+/g, ' ')
16     .trim();
17
18     const isbnRegex = /\b97[89][\-\s]?(?:\d[\-\s]?){9}[\dXx]\b/g;
19     const matches = cleanedText.match(isbnRegex);
20     const combinedResults = [...new Set([...(matches || [])])];
21
22     if (combinedResults.length > 0) {
23         const libriInseriti = [];
24
25         for (const isbnRaw of combinedResults) {
26             const isbn = isbnRaw.replace(/[-\s]/g, "");
27             let result = await pool.query("SELECT * FROM libri WHERE
isbn = $1", [isbn]);
28             let libro;
29
30             if (result.rows.length > 0) {
31                 libro = result.rows[0];
32             } else {
33                 const apiUrl = `https://www.googleapis.com/books/v1/
volumes?q=isbn:${isbn}`;
34                 const response = await fetch(apiUrl);
35                 const data = await response.json();
36
37                 if (!data.items || data.items.length === 0) continue;
38
39                 const info = data.items[0].volumeInfo;
40                 const titolo = info.title || "Sconosciuto";
41                 const autori = info.authors ? info.authors.join(", ") : "
Sconosciuto";
42                 const anno_pubblicazione = info.publishedDate ? parseInt(
info.publishedDate.substring(0, 4)) : null;
43                 const casa_editrice = info.publisher || "Sconosciuta";
44                 const categoria = info.categories ? info.categories.join("
, ") : "Non specificata";
45
46                 await pool.query(
47                     "INSERT INTO libri (isbn, titolo, autori,
anno_pubblicazione, casa_editrice, categoria) VALUES ($1, $2, $3
, $4, $5, $6)",
48                     [isbn, titolo, autori, anno_pubblicazione, casa_editrice

```

```
    , categoria]
    );
50
51    libro = { isbn, titolo, autori, anno_pubblicazione,
    casa_editrice, categoria };
52    }
53
54    libriInseriti.push(libro);
55    }
56
57    res.json({ success: true, isbn_rilevati: combinedResults,
    libri_inseriti: libriInseriti });
58  } else {
59    res.json({ success: false, messaggio: " Nessun ISBN rilevato
    tramite OCR." });
60  }
61
62  fs.unlinkSync(imagePath);
63 } catch (err) {
64   console.error(" Errore nell'elaborazione immagine:", err);
65   res.status(500).send("Errore durante l'elaborazione dell'
    immagine.");
66 }
67 });
```

- **upload.single("image"):** Middleware Multer per singolo file immagine.
- **tesseract.recognize():** OCR basato su rete LSTM che analizza l'immagine e restituisce testo.
- **Rete neurale OCR (LSTM):** Tesseract utilizza una Long Short-Term Memory, rete ricorrente per sequenze di caratteri. Mantiene contesto di riga, distinguendo lettere simili e correggendo errori.
- **Pulizia testo:** Normalizza trattini, rimuove caratteri speciali e spazi multipli.
- **Regex ISBN:** Estrae numeri ISBN dal testo.
- **Database check:** Controlla se ISBN già presente; se assente, richiama API Google Books per dati libro.
- **Inserimento database:** Popola tabella libri con informazioni ottenute.
- **Risposta JSON:** Restituisce ISBN rilevati e libri inseriti.

- **fs.unlinkSync:** Rimuove immagine temporanea dal server.
- **Gestione errori e logging:** Debug OCR e controllo errori lato server.

4.1.10 Rotte del codice:

Rotte

```
1 app.post("/register", async (req, res) => {
2   const { user
3     name, email, password } = req.body;
4   if (!password) return res.status(400).json({ error: "Password
5     mancante" });
6   const hashedPassword = await bcrypt.hash(password, 10);
7   try {
8     await pool.query(
9       "INSERT INTO utente (username, email, password) VALUES ($1, $2
10        , $3)",
11       [username, email, hashedPassword]
12     );
13     res.json({ message: "Registrazione avvenuta con successo!" });
14   } catch (err) {
15     console.error(err);
16     res.status(500).json({ error: "Errore durante la registrazione"
17     });
18   }
19 });
```

- **Rotte e API:** Da qui in poi si definiscono le diverse rotte dell'applicazione, che gestiscono funzionalità come upload di PDF, upload di immagini per OCR, gestione utenti e interazioni con il database.
- **Middleware e logica:** Ogni rotta utilizza middleware appropriati (es. Multer per file, parsing JSON) e include logica per validazioni, query al database e chiamate a servizi esterni come Google Books.
- **Gestione errori e sicurezza:** Tutte le rotte implementano controlli per errori, validazione dei dati e rimozione di file temporanei per sicurezza e performance.

4.2 Lato frontend : index.html

4.2.1 Struttura iniziale della pagina (DOCTYPE, HTML e HEAD)

```
<head>

1 <!DOCTYPE html>
2 <html lang="it">
3 <head>
4 <!-- HEAD della pagina -->
5 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/choices.js
  /public/assets/styles/choices.min.css" />
6 <script src="https://cdn.jsdelivr.net/npm/choices.js/public/assets/
  scripts/choices.min.js"></script>
7
8 <meta charset="UTF-8" />
9 <meta name="viewport" content="width=device-width, initial-scale=1.0
  "/>
10 <title>biblioTECHa</title>
```

Il documento inizia con `<!DOCTYPE html>` per indicare che si tratta di un file HTML5. Il tag `<html lang="it">` specifica che la lingua della pagina è l'italiano.

All'interno del `<head>` si trovano informazioni come metadati, collegamenti a fogli di stile e script, e il titolo della pagina. Il collegamento tramite `<link>` importa i fogli di stile della libreria *Choices.js* per gestire menu a tendina avanzati. Il tag `<script>` importa il file JavaScript della stessa libreria, necessario per le interazioni.

Il meta tag `<meta charset="UTF-8">` definisce la codifica dei caratteri, mentre `<meta name="viewport" content="width=device-width, initial-scale=1.0">` rende la pagina responsive.

Infine, `<title>biblioTECHa</title>` definisce il titolo visualizzato nella scheda del browser.

4.2.2 Definizione degli stili CSS

<style>

```
1 <style>
2 body {
3   font-family: 'Roboto', sans-serif;
4   margin: 0;
5   padding: 0;
6   background-color: #1E1E2F;
7   color: #F5F5F5;
8 }
9
10 h1, h3 {
11   font-family: 'Lora', serif;
12   color: #FFFFFF;
13 }
14 /* Header */
15 header {
16   background-color: #2A2A3D;
17   color: #90CAF9;
18   padding: 30px;
19   text-align: center;
20   border-bottom: 2px solid #333;
21   box-shadow: 0 2px 6px rgba(0,0,0,0.4);
22   font-size: 2.25rem;
23 }
24 /* Container */
25 .container {
26   max-width: 1200px;
27   margin: auto;
28   padding: 40px 20px;
29 }
30 /* Buttons */
31 .buttons {
32   display: flex;
33   flex-wrap: wrap;
34   gap: 12px;
35   margin-bottom: 30px;
36 }
37 ...
38 </style>
```

Il foglio di stile definisce l'aspetto visivo generale della pagina e dei suoi componenti principali.

- **Stile generale:** il corpo della pagina (body) usa il font *Roboto*, margini e padding a zero, sfondo scuro e testo chiaro. I titoli (h1, h3) usano il font *Lora* e sono bianchi.
- **Header:** l'elemento header ha sfondo scuro, testo azzurro chiaro, padding consistente, bordo inferiore e ombra leggera per evidenziare la testata della pagina.
- **Container:** la classe `.container` centra il contenuto con larghezza massima di 1200px e padding interno per organizzare lo spazio.
- **Bottoni:** bottoni colorati con bordi arrotondati e ombre. Effetti al passaggio del mouse e al click tramite transizioni. La classe `.buttons` gestisce layout flessibile e spaziatura.
- **Schede (Cards):** elementi `.card` con sfondo scuro, padding, margini, bordi arrotondati e ombra. Al passaggio del mouse, si sollevano leggermente.
- **Input e Select:** campi di input e selezione con padding, bordi arrotondati, sfondo scuro e testo chiaro. Al focus il bordo cambia colore.
- **Tabelle:** tabelle a tutta larghezza, con sfondo scuro e bordi arrotondati. Intestazioni blu (th) e celle scure (td). Cambiano colore al passaggio del mouse sulle righe.
- **Bottoni Azione e Rimuovi:** bottoni distintivi verde per azioni (`.action-btn`) e rosso per rimuovere (`.remove-btn`) con effetti di hover e click.
- **Barra di Ricerca e Filtri:** barra di ricerca e filtri con sfondo scuro, padding e bordi arrotondati. Il filtro tag (`#tagFilter`) consente selezioni multiple e evidenzia i tag selezionati.
- **Altri elementi:** bottoni utente (`.user-button`) blu con effetto hover leggero; liste (ul li) con sfondo scuro, padding, margini e effetto hover.
- **Responsive Design:** media query per dispositivi con larghezza massima di 768px, modificando header, bottoni, tabelle e container per mantenere leggibilità e usabilità.

4.2.3 Header con il titolo dell'applicazione

<header>

```
1 <header>
2   <h1>biblioTECHa</h1>
3 </header>
```

- **Header:** la sezione `<header>` contiene il titolo principale del sito.
- **Titolo:** all'interno dell'header è presente un elemento `<h1>` con il testo "biblioTECHa", che rappresenta il titolo della pagina e identifica l'applicazione.

4.2.4 Contenitore principale e pulsanti di navigazione

<>

```
1 div class="container">
2   <div class="buttons">
3     <button id="btnLogin" onclick="showLoginForm()">Accedi</button>
4     <button id="btnRegister" onclick="showRegisterForm()">Registrati
5     </button>
6     <button id="btnLogout" onclick="logout()" style="display:none;">
7     Logout</button>
8     <button id="btnPublicLibrary" onclick="makeLibraryPublic()"
9     style="display:none;">Rendi pubblica</button>
10    <button id="btnPrivatLibrary" onclick="makeLibraryPrivat()"
11    style="display:none;">Rendi privata</button>
12    <button id="btnSearchOnline" onclick="showOnlineSearchBar()"
13    style="display:none;">Cerca Online</button>
14  </div>
15 ...
```

Questa sezione del codice HTML definisce la parte interattiva della pagina, ossia i moduli di autenticazione, la gestione della libreria personale e globale, e le funzionalità di ricerca e caricamento dei contenuti. Gli elementi sono organizzati all'interno di un contenitore principale.

- **Container principale:** la sezione `<div class="container">` racchiude i diversi elementi della pagina.
- **Pulsanti di autenticazione:** un gruppo di bottoni gestisce le operazioni di login, registrazione, logout e impostazioni di visibilità della libreria (pubblica/privata). Al-

cuni pulsanti sono inizialmente nascosti e vengono mostrati dinamicamente tramite JavaScript.

- **Form di accesso e registrazione:** due card (`loginForm` e `registerForm`) contengono i campi di input per email, username e password, oltre ai rispettivi pulsanti per l'invio.
- **Home utente:** una card visibile solo dopo l'accesso, che mostra un messaggio di benvenuto e tre pulsanti per navigare tra la libreria personale, globale e degli utenti.
- **Gestione ISBN:** sezione che consente l'inserimento manuale di un codice ISBN oppure il caricamento di un'immagine contenente il codice a barre, tramite un form di upload.
- **Filtri di ricerca:** barra di ricerca testuale per titolo/autore/genere e selettore multiplo di tag (`tagFilter`) per filtrare i libri in tabella.
- **Popup inserimento risorse:** finestra modale per aggiungere un link ad un ebook o caricare un file PDF. Comprende campi di input, pulsanti di salvataggio e annullamento.
- **Visualizzazione libreria:** sezione `libraryDisplay` dove vengono mostrati i libri presenti.
- **Ricerca online:** blocco `searchOnlineContainer`, inizialmente nascosto, che permette di cercare libri online per titolo, autore o ISBN. I risultati vengono caricati dinamicamente in `onlineResults`.

4.2.5 Script JavaScript: logica e interattività della pagina

```
<script>

1 <script>
2   let user = null;
3   let currentView = "";
4   let currentUserId = null;
5
6   document.addEventListener("DOMContentLoaded", () => {
7     document.getElementById('searchTable').style.display = 'none';
8     document.getElementById('tagFilterContainer').style.display =
9       'none';
10  });
11  function showLoginForm() {
12    document.getElementById("loginForm").style.display = "block";
13    document.getElementById("registerForm").style.display = "none"
14  };
15
16  ...
17
18  async function login() {
19    const email = document.getElementById("emailLogin").value;
20    const password = document.getElementById("passwordLogin").value;
21
22    const response = await fetch('/login', {
23      method: 'POST',
24      headers: { 'Content-Type': 'application/json' },
25      body: JSON.stringify({ email, password })
26    });
27    ...
28  </script>
29 </body>
30 </html>
```

Lo script JavaScript incluso nel progetto ha il compito di gestire l'interattività della pagina e le operazioni lato client. In particolare implementa le funzioni per:

- gestire il login, la registrazione e il logout degli utenti;
- modificare la visibilità della libreria (pubblica o privata);

- gestire la visualizzazione delle diverse sezioni (form di accesso, libreria personale, libreria globale, libreria utenti);
- effettuare la ricerca di libri tramite ISBN, caricamento di immagini o inserimento manuale;
- filtrare i libri per titolo, autore, genere o tag;
- aggiungere risorse esterne come link a ebook o file PDF caricati;
- eseguire ricerche online e visualizzarne i risultati;
- aggiornare dinamicamente il contenuto della libreria (`libraryDisplay`).

Capitolo 5

Modelli Deep per il riconoscimento di codici ISBN

5.1 Introduzione al Deep Learning

L'apprendimento profondo o deep learning (DL), ha rappresentato una rivoluzione nel campo dell'intelligenza artificiale, distinguendosi dagli approcci tradizionali di machine learning per la sua capacità di apprendere rappresentazioni dei dati attraverso multiple stratificazioni di elaborazione [1]. Il termine "deep" si riferisce alla presenza di multipli layer che caratterizzano l'architettura del modello, chiamato rete neurale (NN).

Ognuno di questi layer ha un compito preciso. Possono trasformare le feature dei dati in maniera gerarchica e progressivamente più astratta.

A differenza dei metodi classici, che richiedono spesso un feature engineering manuale, il deep learning automatizza questo processo, permettendo al modello di apprendere direttamente dai dati grezzi le caratteristiche più rilevanti. Questa capacità è resa possibile dalla composizionalità delle reti neurali profonde, dove i layer iniziali catturano pattern elementari, mentre quelli successivi combinano queste informazioni per formare concetti più complessi [3].

Ad oggi, i modelli di DL sono utilizzati in numerosi campi d'applicazione con l'utilizzo di tecniche specifiche all'applicazione. Le Convolutional Neural Networks (CNN, [11]) per l'elaborazione e il riconoscimento di immagini, le Recurrent Neural Networks (RNN, [12]) per sequenze temporali o ancora gli svariati approcci dei modelli generativi (VAE, GAN, Flow Models) () sono solo alcuni degli svariati approcci DL.

5.2 Riconoscimento Ottico dei Caratteri (OCR)

Il Riconoscimento Ottico dei Caratteri (Optical Character Recognition, OCR) costituisce una tecnologia fondamentale per la digitalizzazione e l'elaborazione automatica di documenti. La sua evoluzione ha attraversato diverse fasi storiche, dai primi sistemi basati su template matching fino agli approcci moderni basati sul deep learning [2].

I sistemi OCR tradizionali operavano attraverso una pipeline sequenziale che includeva fasi di pre-processing, segmentazione dei caratteri, estrazione di feature e classificazione. Questi sistemi, sebbene efficaci per font standard e condizioni controllate, mostravano limitazioni significative nella gestione di variabilità tipografiche e layout complessi.

Con l'avvento del Deep Learning, il paradigma dell'OCR ha subito una trasformazione radicale. I moderni sistemi tendono verso architetture *end-to-end* che integrano tutte le fasi del processo in un unico modello neurale. Questo approccio consente di superare molte delle limitazioni dei metodi tradizionali, specialmente nella gestione di tre categorie di testi particolarmente complesse: caratteri decorativi, testo curvilineo e documenti storici degradati [10].

I **caratteri decorativi** rappresentano una sfida per gli algoritmi di riconoscimento, in quanto le lettere presentano variazioni stilistiche, ornamentazioni e legature che si discostano notevolmente dai font standard su cui i modelli vengono solitamente addestrati. Questi elementi grafici possono compromettere la segmentazione e la classificazione dei caratteri, rendendo necessario un modello capace di generalizzare su una varietà ampia di stili tipografici.

Il **testo curvilineo**, invece, pone difficoltà legate alla disposizione non lineare delle parole. Nei metodi tradizionali, l'OCR presuppone una struttura testuale allineata orizzontalmente; tuttavia, in molti contesti reali (ad esempio loghi, insegne o grafica editoriale) il testo può seguire percorsi curvi o irregolari. Le architetture basate su reti neurali convoluzionali e ricorrenti, o più recentemente su *transformer*, permettono di modellare queste deformazioni geometriche e mantenere la coerenza semantica lungo la curva del testo.

Infine, i **documenti storici degradati** rappresentano un ulteriore livello di complessità dovuto alla presenza di rumore, macchie, inchiostro sbiadito o supporti cartacei deteriorati. In tali casi, i modelli di OCR devono essere in grado di estrarre informazioni da immagini con basso contrasto e alta variabilità, spesso senza una segmentazione chiara tra testo e sfondo. L'integrazione di reti neurali profonde consente di apprendere rappresentazioni robuste alle distorsioni e di recuperare informazioni testuali anche in condizioni di forte degrado visivo.

Le sfide contemporanee nell'OCR includono la gestione di documenti multilingua, la preservazione della struttura semantica del testo, il riconoscimento di formule matematiche e l'elaborazione efficiente di grandi volumi documentali.

5.2.1 Tesseract OCR

Tesseract rappresenta uno dei motori OCR open source più noti e storicamente significativi nel panorama del riconoscimento del testo [5]. La sua storia inizia nei laboratori Hewlett-Packard (HP) tra il 1985 e il 1995, dove fu sviluppato come motore proprietario per la scannerizzazione [16]. Un momento di svolta cruciale per il progetto si ebbe nel 2006, quando Google assunse lo sviluppo e lo rilasciò come software open source [15]. Sotto la guida di Google, Tesseract ha compiuto una transizione fondamentale, abbandonando le tecniche di pattern matching tradizionali per abbracciare pienamente, a partire dalla versione 4, architetture di riconoscimento basate su reti neurali LSTM (Long Short-Term Memory), mantenendo così la sua rilevanza nel panorama moderno dell'OCR.

La versione 4.x di Tesseract ha introdotto il supporto per le reti LSTM (Long Short-Term Memory), un tipo di RNN particolarmente efficace nell'elaborazione di sequenze [4]. Questo cambiamento architetturale ha segnato un miglioramento significativo nelle prestazioni, specialmente per quanto riguarda la capacità di generalizzazione su font e lingue diverse.

La pipeline di elaborazione di Tesseract segue un flusso strutturato che combina tecniche tradizionali di image processing, il processo include pre-processing dell'immagine, analisi del layout, riconoscimento tramite motore LSTM e post-elaborazione con correzione linguistica.

Uno degli aspetti più rilevanti di Tesseract è il suo supporto multilingua, che comprende attualmente oltre 100 lingue differenti. Le configurazioni permettono un fine tuning attraverso i Page Segmentation Modes (PSM), che ottimizzano il comportamento per scenari specifici come testo singolo colonna o singole parole.

Nonostante i progressi, Tesseract presenta limitazioni in scenari complessi quali testo su sfondi texture, immagini con risoluzione subottimale e documenti con layout non convenzionali. L'integrazione in applicazioni web è resa possibile attraverso Tesseract.js, che mantiene la maggior parte delle funzionalità del motore originale in ambienti browser-based.

5.3 Diagnosi delle Criticità

5.3.1 Approccio con Rete Neurale Personalizzata

Nella fase iniziale del progetto, prima di adottare l'utilizzo di una tesseract, è stato implementato e addestrato un modello di DL utilizzando il framework PyTorch [17].

Questo approccio, sebbene formativo dal punto di vista dell'apprendimento, ha rivelato diverse criticità fondamentali che ne hanno pregiudicato l'efficacia pratica per il compito di riconoscimento ottico dei caratteri applicato ai codici ISBN.

Scarsa capacità di generalizzazione e overfitting

Il modello di rete neurale convoluzionale sviluppato, costituito da due livelli convoluzionali seguiti da due strati fully connected, ha mostrato una marcata tendenza all'overfitting. Durante la fase di addestramento, la rete riusciva a riconoscere quasi perfettamente le immagini presenti nel dataset di training, ma falliva sistematicamente nel generalizzare tale conoscenza a nuovi esempi di ISBN appartenenti al set di validazione o di test.

Piccole variazioni nelle immagini di input, come leggere rotazioni, sfocature, variazioni di luminosità o contrasto, risultavano sufficienti a compromettere completamente la capacità di classificazione. Questo comportamento suggerisce che il modello non abbia appreso caratteristiche visive generali e invarianti, come bordi o forme dei caratteri, ma piuttosto abbia memorizzato i pattern specifici dei pixel delle immagini viste in fase di addestramento. Tale fenomeno è riconducibile a una combinazione di fattori, tra cui la limitata quantità di dati disponibili e la relativa complessità del modello rispetto al compito da svolgere, portando a un apprendimento eccessivamente specifico [1].

Insufficienza quantitativa e qualitativa dei dati

Il dataset utilizzato, organizzato in sottocartelle corrispondenti a ciascun ISBN, presentava un numero di campioni per classe statisticamente insufficiente per garantire un addestramento efficace di una rete neurale. Il ridotto numero di immagini per ogni ISBN non ha consentito al modello di apprendere la naturale variabilità del dominio visivo, rendendolo estremamente sensibile anche a minime alterazioni.

Inoltre, la qualità delle immagini non sempre risultava omogenea: differenze in illuminazione, messa a fuoco e angolazione hanno introdotto ulteriore rumore nei dati. L'impiego di tecniche di *data augmentation* — come rotazioni casuali, traslazioni, modifiche della luminosità e della saturazione — ha contribuito ad ampliare artificialmente la varietà del dataset, ma non è stato sufficiente a compensare la scarsità di dati reali. Come mostrato nelle figure 5.1–5.3, tali trasformazioni hanno migliorato marginalmente la robustezza del modello, ma non in misura tale da prevenire il fenomeno di overfitting [6].

Modifica la luminosità dell'immagine ↗

Modifica la luminosità dell'immagine con `tf.image.adjust_brightness` fornendo un fattore di luminosità:

```
bright = tf.image.adjust_brightness(image, 0.4)
visualize(image, bright)
```



Figura 5.1: Tecnica di data augmentation: variazione della luminosità [18]

Satura un'immagine

Saturare un'immagine con `tf.image.adjust_saturation` fornendo un fattore di saturazione:

```
saturated = tf.image.adjust_saturation(image, 3)
visualize(image, saturated)
```



Figura 5.2: Tecnica di data augmentation: modifica della saturazione [18]

Scala di grigi un'immagine ↗

Puoi ridimensionare un'immagine in scala di grigi con `tf.image.rgb_to_grayscale` :

```
grayscaled = tf.image.rgb_to_grayscale(image)
visualize(image, tf.squeeze(grayscaled))
_ = plt.colorbar()
```

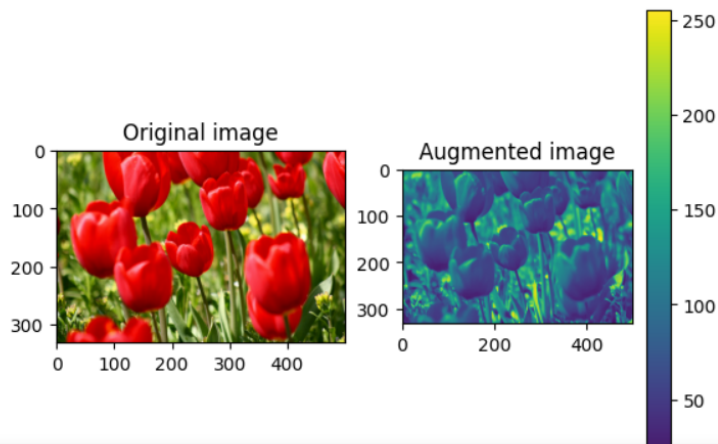


Figura 5.3: Tecnica di data augmentation: conversione in scala di grigi [18]

Problematiche nel preprocessing e normalizzazione delle immagini

Le immagini del dataset presentavano dimensioni e proporzioni eterogenee, generando errori critici durante la fase di caricamento dei batch e nella successiva propagazione dei tensori attraverso la rete. Poiché gli strati fully connected della CNN richiedono vettori di input a dimensione fissa, si è reso necessario un ridimensionamento forzato di tutte le immagini a 64×64 pixel.

Tale operazione, tuttavia, ha comportato la distorsione dell'aspect ratio originale, deformando spesso la struttura dei caratteri numerici e dei trattini che compongono l'ISBN. Di conseguenza, la rete ha appreso rappresentazioni distorte dei pattern testuali, riducendo ulteriormente la capacità di generalizzazione. La normalizzazione delle immagini, basata sui valori medi e deviazioni standard di ImageNet, sebbene corretta dal punto di vista tecnico, non ha potuto compensare le perdite informative introdotte dal ridimensionamento [3].

Architettura del modello sovradimensionata e inefficiente

L'architettura del modello, pur essendo composta da soli due strati convoluzionali e due fully connected, si è rivelata sovradimensionata rispetto alla complessità effettiva del compito. Il problema affrontato, ossia la classificazione di immagini contenenti un codice numerico

stampato, non richiede una rete con un numero così elevato di parametri, specialmente in presenza di un dataset di dimensioni ridotte.

Durante l'addestramento, ogni epoca richiedeva tempi computazionali considerevoli a fronte di un miglioramento marginale della loss, evidenziando un rapporto inefficiente tra capacità di rappresentazione e disponibilità di dati. Il modello ha pertanto mostrato un'eccessiva flessibilità, tendendo a memorizzare il rumore visivo piuttosto che a estrarre le caratteristiche rilevanti dei caratteri numerici, con conseguente peggioramento delle prestazioni generali.

Limite concettuale del sistema di etichettatura

Il sistema di supervisione adottato, basato su un file di mappatura `idx_to_class.json` che associava a ciascuna immagine una singola etichetta corrispondente a un intero ISBN, ha introdotto un limite concettuale importante. Il modello, infatti, non è stato progettato per leggere e interpretare le singole cifre del codice, ma per assegnare l'intera immagine a una classe predefinita.

In tale configurazione, ogni ISBN veniva trattato come una classe indipendente, rendendo impossibile il riconoscimento di un codice non visto in fase di addestramento. Questo approccio non risulta scalabile e non rispecchia la logica di un vero sistema OCR, che dovrebbe essere in grado di riconoscere sequenze di caratteri alfanumerici indipendentemente dal loro contenuto. Il limite strutturale di questa impostazione ha rappresentato una delle principali motivazioni che hanno portato all'abbandono dell'approccio CNN puro in favore di strumenti OCR tradizionali.

5.3.2 Performance di Tesseract OCR

Per i motivi elencati sopra, abbiamo adottato Tesseract OCR per-addestrato per il riconoscimento ottico dei caratteri. Nonostante il pre-addestramento abbiamo notato alcune criticità durante la fase di testing su campioni reali. In particolare, si è osservata un'affidabilità inconsistente del riconoscimento, con una notevole variabilità nelle performance dove alcune immagini venivano processate correttamente mentre altre producevano output completamente errati o parziali [2]. In test su 50 immagini reali, Tesseract ha identificato correttamente l'ISBN nel circa 70% dei casi, fallendo nel 20% e producendo risultati parziali nel 10% restante.

Il sistema si è rivelato particolarmente sensibile alle condizioni di acquisizione, mostrando significativi problemi in presenza di angolazioni non ottimali, riflessi e ombre sulle copertine, variabilità nella qualità e risoluzione dell'immagine, e dimensioni ridotte o poco contrastate del testo ISBN. Le limitazioni intrinseche di Tesseract hanno ulteriormente influenzato negativamente la precisione del riconoscimento, con una whitelist di caratteri troppo restrit-

tiva, un page segmentation mode non ottimale per testi brevi come l'ISBN, e una scarsa robustezza nei confronti di font artistici o stilizzati [7]. Durante i test sono state identificate ulteriori criticità specifiche, tra cui la tendenza di Tesseract a estrarre testo irrilevante privilegiando il testo descrittivo principale rispetto ai codici numerici, la difficoltà nell'identificare la regione specifica contenente il codice tra il resto del testo, problemi nel riconoscere sequenze numeriche interrotte da spazi o caratteri speciali, e performance degradate su copertine con elementi grafici multipli [8].

Caso di studio concreto: analisi di un errore

Un esempio emblematico dei limiti del sistema è rappresentato dall'elaborazione mostrata in figura, dove il sistema OCR ha prodotto un output errato. Nell'analisi di questo caso specifico emergono diverse problematiche: l'OCR ha correttamente riconosciuto il corpo principale del testo ma ha trascurato la sequenza numerica dell'ISBN, ha elaborato in modo frammentario l'ultima riga riconoscendo parzialmente i numeri ma senza identificarne la struttura corretta, e non è stato in grado di ricostruire la sequenza completa validandola appropriatamente [9]. Inoltre, la disposizione su più colonne e la presenza di elementi grafici ha confuso l'algoritmo di segmentazione della pagina.

```
Testo OCR rilevato: Un libro che esalta l'amore e la musica in un rigurgito d
oman-
ticismo di cui l'autore sembra a buon diritto l'epigono rappre-
sentante.
Il suo stile, vagamente surreale, ripercorre fedelmente l'itinerario
storico legato al contesto musicale e ai suoi personaggi reali.
Questi ultimi grandi compositori vissuti a Vienna e annoverati in
un intento narrativo raffinato e culturalmente consapevole.
Nel quale quindi essi rivivono con le loro opere dando vita anche
a uno scenario bene articolato e appropriato allo svolgersi della
trama del romanzo, in cui non mancano certo episodi sentimentali
non comuni e perciò bisognosi di partecipazione da parte del
lettore.

| | sex a
mr
» ' e y
JES Puro 13.00 7 AR 9 17888991950 156)
```

Figura 5.4: Esempio concreto di errore OCR: il sistema ha estratto il testo descrittivo ma non ha identificato correttamente l'ISBN presente nell'immagine

Capitolo 6

Discussione

6.1 Introduzione

Questo capitolo delinea le strategie di miglioramento proposte per il sistema bibliotecario sviluppato, affrontando le limitazioni emerse durante le fasi di testing e sviluppo. Le proposte si articolano in quattro aree principali: ottimizzazione del modulo OCR, potenziamento della sicurezza, avanzamento degli algoritmi di ricerca e miglioramento dell'esperienza utente. Ogni sezione presenta soluzioni concrete che potrebbero essere implementate in future iterazioni del progetto.

6.2 Strategie di miglioramento proposte per OCR

Per affrontare le criticità identificate, si propone un approccio multi-livello che includa un pre-processing avanzato delle immagini con implementazione di filtri per l'enhancement del contrasto nelle regioni numeriche, ritaglio selettivo delle aree tipiche di posizionamento ISBN, e binarizzazione adattiva per isolare il testo dallo sfondo [10]. È inoltre necessaria un'ottimizzazione della configurazione Tesseract attraverso la sperimentazione con diversi page segmentation mode, la configurazione di whitelist mirate per sequenze numeriche, e l'adjustamento dei parametri di confidence per filtrare risultati incerti [2]. Un'elaborazione multi-pass che combini prima una elaborazione generale dell'intera immagine, poi una elaborazione mirata sulle regioni di interesse, e infine una fusione intelligente dei risultati con sistema di voting, potrebbe significativamente migliorare l'affidabilità del riconoscimento [8].

Questo caso dimostra la necessità di un approccio più sofisticato che combini tecniche tradizionali di OCR con euristiche specifiche per il dominio degli ISBN, garantendo una maggiore affidabilità nel riconoscimento di sequenze numeriche critiche per l'applicazione.

6.3 Sicurezza e Autenticazione Avanzata

Partendo dall'attuale implementazione di bcrypt per l'hashing delle password, che ha dimostrato robustezza durante il testing, si propone di estendere il sistema con meccanismi di autenticazione a due fattori. Attualmente l'applicazione utilizza un approccio tradizionale basato su password hashate con bcrypt, che sebbene solido, potrebbe essere integrato con tecniche più avanzate per rispondere alle minacce moderne.

L'implementazione dell'autenticazione a due fattori (2FA) rappresenterebbe un significativo passo avanti nella protezione degli account, specialmente per gli utenti amministratori che gestiscono funzionalità critiche del sistema. Questo meccanismo potrebbe essere realizzato tramite applicazioni authenticator come Google Authenticator o Authy, generando codici temporanei monouso (TOTP) che aggiungono un ulteriore livello di sicurezza oltre alla password. In alternativa, per utenti meno tecnici, si potrebbe valutare l'invio di codici via SMS, sebbene questo approccio presenti vulnerabilità legate alla sicurezza delle reti mobili.

Per quanto concerne la gestione delle sessioni, l'adozione di token JWT (JSON Web Tokens) con meccanismi di refresh automatico consentirebbe di bilanciare sicurezza e usabilità. I token di accesso potrebbero avere durata limitata (15-30 minuti), mentre token di refresh più longevi permetterebbero di mantenere l'utente autenticato senza compromettere la sicurezza. Questo approccio andrebbe integrato con un sistema di controllo degli accessi basato su ruoli (RBAC), dove differenti privilegi sono assegnati in base al profilo utente (amministratore, utente registrato, visitatore).

La validazione degli input rappresenta un altro fronte importante di miglioramento. Oltre alle attuali verifiche, si potrebbe implementare una sanitizzazione completa dei dati in ingresso, utilizzando librerie specializzate per prevenire attacchi XSS (Cross-Site Scripting) e SQL injection. Un approccio proattivo prevedrebbe l'analisi statica del codice e test di penetrazione periodici per identificare vulnerabilità potenziali prima che vengano sfruttate maliciosamente.

6.4 Algoritmi di Ricerca e Raccomandazione

Le funzionalità di ricerca e scoperta dei contenuti rappresentano un ambito con significativo potenziale di miglioramento. L'attuale sistema di ricerca, sebbene funzionale, potrebbe beneficiare dell'integrazione di Elasticsearch per implementare ricerche full-text avanzate. Questo motore di ricerca sarebbe in grado di gestire sinonimi, correggere errori di battitura comuni e comprendere la semantica del dominio bibliografico, restituendo risultati più pertinenti anche quando le query non corrispondono esattamente ai termini nel database.

Un sistema di raccomandazione intelligente costituirebbe un valore aggiunto notevole per l'esperienza utente. Basandosi su algoritmi di collaborative filtering, il sistema potrebbe analizzare il comportamento di lettura degli utenti con interessi simili e suggerire titoli rilevanti. Questo meccanismo potrebbe essere potenziato con tecniche di content-based filtering, che considerano le caratteristiche intrinseche dei libri (genere, autore, anno di pubblicazione) e il profilo di preferenze di ciascun utente. L'implementazione di tali algoritmi richiederebbe la raccolta di dati di interazione (libri visualizzati, aggiunti alla libreria, ricerche effettuate) e l'elaborazione attraverso framework di machine learning.

Completare questo ecosistema di discovery, una dashboard amministrativa per l'analisi dei dati fornirebbe insights preziosi sull'utilizzo della piattaforma. Metriche come i libri più popolari, gli utenti più attivi, i pattern di ricerca più frequenti e i tassi di conversione delle raccomandazioni permetterebbero di prendere decisioni basate sui dati per ottimizzare continuamente il servizio.

6.5 User Experience e Interfaccia

L'evoluzione dell'interfaccia utente verso un'esperienza più moderna e inclusiva rappresenta una direzione strategica per il progetto. La trasformazione dell'applicazione in una Progressive Web App (PWA) consentirebbe di unire i vantaggi delle applicazioni web e mobile native. Gli utenti potrebbero installare l'applicazione sul proprio dispositivo, accedervi in modalità offline per consultare la libreria personale già scaricata, e ricevere notifiche push per aggiornamenti sui nuovi contenuti disponibili. Questo approccio migliorerebbe significativamente l'engagement e la retention degli utenti mobili.

La creazione di un design system coerente e ben documentato garantirebbe consistenza visiva e funzionale across tutta l'applicazione. Componenti UI riutilizzabili per elementi comuni come card dei libri, barre di ricerca, pulsanti e modali non solo accelererebbero lo sviluppo di nuove funzionalità, ma assicurerebbero un'esperienza utente prevedibile e intuitiva. Il design system dovrebbe includere linee guida per typography, color palette, spaziatura e micro-interazioni, facilitando anche il lavoro di eventuali nuovi sviluppatori che si unissero al progetto.

L'accessibilità rappresenta un aspetto etico e legale fondamentale spesso trascurato. Implementare il supporto completo per screen reader e la navigazione da tastiera, in conformità con le linee guida WCAG 2.1 livello AA, renderebbe l'applicazione utilizzabile da persone con disabilità visive o motorie. Questo includerebbe l'uso appropriato di attributi ARIA, il contrasto cromatico sufficiente, la gestione corretta del focus della tastiera e alternative testuali per tutti gli elementi non testuali. Un'applicazione accessibile non solo amplia la base utente, ma dimostra l'attenzione del progetto all'inclusività digitale.

6.6 Conclusioni e Sviluppi Futuri

Le proposte presentate in questo capitolo delineano un percorso di evoluzione che, se implementato, trasformerebbe l'attuale prototipo in un sistema production-ready. La priorità dovrebbe essere data al potenziamento del modulo OCR, seguito dal rafforzamento della sicurezza e infine dal miglioramento dell'esperienza utente. Questi sviluppi richiederebbero un'analisi costi-benefici approfondita, ma promettono di elevare significativamente il valore dell'applicazione nel contesto della gestione bibliotecaria digitale.

Bibliografia

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
Link : <https://www.deeplearningbook.org/>.
- [2] Smith, R. (2007). *An overview of the Tesseract OCR engine*. Ninth International Conference on Document Analysis and Recognition.
Link : <https://ieeexplore.ieee.org/abstract/document/4376991>.
- [3] LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. Nature, 521(7553), 436-444.
Link : <https://www.nature.com/articles/nature14539>.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural computation, 9(8), 1735-1780.
Link : <https://ieeexplore.ieee.org/abstract/document/6795963>.
- [5] Tesseract OCR (2023). *Tesseract OCR Documentation*. <https://github.com/tesseract-ocr/tesseract>
- [6] Shorten, C., & Khoshgoftaar, T. M. (2019). *A survey on Image Data Augmentation for Deep Learning*. Journal of Big Data, 6(1), 1-48.
Link : <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0>.
- [7] Antonacopoulos, A., Bridson, D., Papadopoulos, C., & Pletschacher, S. (2009). *A realistic dataset for performance evaluation of document layout analysis*. 10th International Conference on Document Analysis and Recognition, 296-300.
- [8] Shafait, F., Keysers, D., & Breuel, T. M. (2008). *Performance evaluation and benchmarking of six-page segmentation algorithms*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 30(6), 941-954.
- [9] Rizvi, S., Pfeiffer, T., & Poovendran, R. (2011). *Book cover identification for library inventory management using mobile computers*. 2011 IEEE International Conference on Pervasive Computing and Communications Workshops, 610-615.

- [10] Ye, Q., & Doermann, D. (2017). *Text detection and recognition in imagery: A survey*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 37(7), 1480-1500.
- [11] Stanford University (2024). *Convolutional Neural Networks (CNNs / ConvNets)*. CS231n: Deep Learning for Computer Vision Course Notes.
Link : <http://cs231n.github.io/convolutional-networks/>.
- [12] Olah, C. (2015). *Understanding LSTMs*. Colah's Blog.
Link : <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- [13] Weng, L. (2021). *What are Diffusion Models?*. Lil'Log.
Link : <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>.
- [14] Weng, L. (2018). *Flow-based Deep Generative Models*. Lil'Log.
Link : <https://lilianweng.github.io/posts/2018-10-13-flow-icml/>.
- [15] Tesseract History. *Tesseract OCR: Now we have it*. [Wiki page].
Link: [https://it.wikipedia.org/wiki/Tesseract_\(software\)](https://it.wikipedia.org/wiki/Tesseract_(software))
- [16] HP history. *History of Hewlett Packhard*. [Wiki page].
Link: <https://en.wikipedia.org/wiki/Hewlett-Packard>.
- [17] PyTorch History. *The Complete History and Evolution of PyTorch*.
Link: <https://tensorgym.com/blog/pytorch-history>
- [18] Data Augmentation. *Image augmentation*. Tensorflow. Link :
https://www.tensorflow.org/tutorials/images/data_augmentation?hl=it
- [19] PostgreSQL. *Navigating the database landscape*.
Link : <https://xata.io/blog/sql-mysql-postgresql-nosql>

“Enjoy the butterflies, enjoy being naïve.

Enjoy the nerves, the pressure, people not knowing your name.

Enjoy the process of making a name for yourself, getting faster and faster with each lap and meeting some great people along the way.

Bring friends along. Bring family along. Don’t assume they’ll be a distraction. Don’t be afraid to surround yourself with people you care about and love.”

— Danny Ric